A RNN Decoder for Channel Decoding under Correlated Noise

Xiangxiang Zhang, Tao Luo
Beijing Key Laboratory of Network System Architecture and Convergence
Beijing University of Posts and Telecommunications
Beijing, China
{zxx0551&tluo}@bupt.edu.cn

***Abstract*—We present and use a recurrent neural network as RNN decoder for the convolutional code under correlated noise. Using the bidirectional GRU(Gated Recurrent Unit) network and the fully connected neural network(DNN), the sequence feature information of the convolutional code is extracted, and the decoding result is calculated by the fully connected neural network. The RNN neural network decoder achieves better BER(Bit Error Rate) performance than the traditional Viterbi decoding algorithm for convolutional codes with shorter memory lengths, such as (2, 1, 3) convolutional codes under correlated noise. The greater noise correlation, the greater performance improvement of the decoder relative to the traditional Viterbi decoding algorithm. Besides, the RNN decoder is close to the MAP performance under the AWGN channel. In addition, the decoder is robust under different noise correlation models. Due to the limitation of the structure and complexity of the RNN decoder, as the memory length of the convolutional code increases, its decoding performance is gradually reduced, which is not suitable for convolutional codes with long memory lengths.**

***Index Terms*—RNN decoder, GRU, convolutional code, correlated noise**

I. Introduction

Channel coding aims to increase transmission reliability by preventing interference. The convolutional code generates parity symbols through a boolean polynomial function. The traditional Viterbi algorithm is focused on the AWGN channel and can reach Maximum Likelihood performance. However, the correlation always exists in the real channel, caused by filtering, oversampling [1], and device noise [2]. Colored noise is divided into many types, and widely exists in mobile objects [3], electronic equipment [4] and common buildings. The Viterbi algorithm degrades performance under correlated noise [5]. In general, the matrix multiplication method is used for white noise. The method requires enough prior knowledge about noise and is just suitable for the single correlation model. Besides, the equivalent coded symbols after this method has a different structure from the transmitted symbols, which can't be used directly for decoding. Thence, the universal and flexible decoding structure for convolutional codes under correlated noise is desired. Nowadays, deep learning provides new ideas and potential to solve physical layer challenges [6]. We provide a summary of these works in Section II.

In this paper, we design the RNN decoder architecture to deal with the decoding problem when correlation exists in channel noise. Instead of processing the correlated noise separately, the RNN decoder directly decodes the received symbols. The RNN decoder concatenates 2 layers of GRU network with 1 layer fully connected network, and the received symbols are directly processed by RNN decoder to obtain the decoding results. The GRU network can remove the interference caused by correlated noise. Then, the fully connected network obtains final decoding results from the information after GRU. The decoding performance of the decoder mainly depends on the GRU's denoising of received symbols and the extraction of input sequence symbols, hence we call it RNN decoder.

The RNN decoder has many properties. Firstly, it performs better than the traditional Viterbi algorithm decoder. Besides, the RNN decoder learns the network directly from the training data, which does not require knowledge from the channel and additional ways. Then, the RNN decoder structure is flexible. As for different correlation models, we can obtain the new training data and retrain the new decoders. Unlike the difficulty of data acquisition in images and texts, we can generate as many training samples as we like through man-made signals.

II. Related Work

A. RNN and GRU Network

In a traditional neural network, we assume that all inputs (and outputs) are independent of each other. However, the order and relevance of the sequence have to be considered in many cases. RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being depended on the previous computations. We can also consider RNNs have a memory which captures information about what has been calculated so far. Here is what a typical RNN looks like:

Figure.1 shows a RNN being unrolled (or unfolded) into a full network. By unrolling we simply mean that we write out the network for the complete sequence. For example, if the sequence we care about is a sentence of n-words, the network would be unrolled into an n-layer neural network, one layer for each word. x_t is the input at time step $t.s_t$ is the hidden

This work is supported in part by the National Natural Science Foundation of China under Grant No. 61571065 and the Beijing Natural Science Foundation (L161005).

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

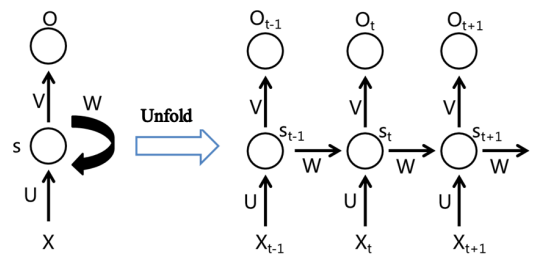


Fig. 1. RNN network structure [7].

state at time step t, which is calculated based on the previous hidden state and the input at the current step. o_t is the output at step t.

Bidirectional RNNs are based on the idea that the output at time t may not only depend on the previous elements in the sequence, but also future elements. There are two RNNs stacked on top of each other. The output is then computed based on the hidden state of both RNNs. In theory, RNNs can make use of information in arbitrarily long sequences, but in practice, they are limited to looking back only a few steps (more on this later). GRU is proposed to solve the problem above.

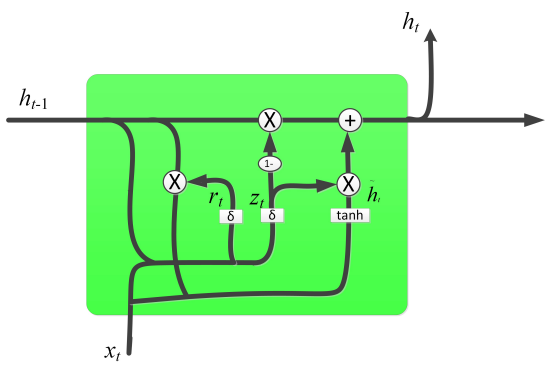


Fig. 2. GRU network structure [7].

GRU [7], first used in 2014. Figure.2 shows the GRU network structure. A GRU has two gates, a reset gate r, and an update gate z. Intuitively, the reset gate determines how to combine the new input with the previous memory, and the update gate defines how much of the previous memory to keep around. If we set the reset gate to all 1's and update gate to all 0's, we again arrive at our plain RNN model. The calculation equations are shown below:

$$\begin{aligned} r_t &= \sigma(W_r \cdot h_{t-1} + U_r \cdot x_t) \\ z_t &= \sigma(W_z \cdot h_{t-1} + U_z \cdot x_t) \\ \tilde{h}_t &= \tanh(W_{\tilde{h}} \cdot (r_t * h_{t-1}) + U_{\tilde{h}} \cdot x_t) \\ h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \\ O_t &= \sigma(W_o \cdot h_t) \end{aligned} \tag{1}$$

where $\cdot$ means dot product and $*$ means direct multiplication.

B. Batch Normalization Layer

In general, the training efficiency of mini-batch stochastic gradient descent (SGD) is largely reduced by internal covariate shift. Batch normalization (short for BN) is proposed to alleviate the internal covariate shift by incorporating a normalization step and a scale and shift step before the nonlinearity in each layer [8]. For batch normalization, only two parameters per activation are added, and they can be updated with back-propagation. Batch normalization performs well at fast training, better performance, and low sensitivity to initialization. In the structure of RNN decoder, we use a BN layer after each layer of GRU to accelerate the convergence of network, which can achieve better decoding performance.

C. Deep Learning for Communication

O'Shea et al propose a new idea to design a communication system for the physical layer through the deep learning [9]. H. He et al apply Learned Denoising-based Approximate Message Passing (LDAMP) network based on DnCNN to the channel estimation for beamspace mmWave massive MIMO system [10]. The deep learning-based channel decoding shows a competitive performance [11] [12] [13]. Fei Liang et al [14] use CNN network to design an iterative BP-CNN architecture for decoding LDPC code under correlated noise. In addition, deep learning is applied to Massive MIMO CSI feedback [15], communication over the air [16] and enables the optimization of the transceiver in a single end-to-end process [17].

The early studies do not provide an effective solution to the performance degradation problem of traditional convolutional code decoding algorithms under correlated noise. In this paper, we design the RNN decoder to improve decoding performance.

III. SYSTEM MODEL

In this section, we introduce the RNN decoder in detail. In order to show our system clearly, we will present our system framework firstly. Besides, the standard noise correlation model is explained in order to demonstrate our system model comprehensively. Then, the RNN decoder parameter setting and training of the network will be introduced.

A. System Framework

Figure.3 illustrates the proposed channel decoding structure with RNN decoder. At the transmitter, the convolution code is defined as (n, k, m) code with k input bits, n output bits, and m constraint length. The uniformly distributed source bits u of length k is encoded into bits s of length n through the convolution code encoder. Then, the bits s is mapped to a symbol vector x through the BPSK modulation. The symbol x will pass through the Gaussian additive correlated noise channel.

The convolutional codes such as (2, 1, 3) and (2, 1, 7) convolutional codes have small receive dimensions, so we apply the temporal dimension to the correlated noise in our model. The correlated noise can be generated by white noise and is denoted as $n_c = n_w C^{1/2}$ where n_w is an $n \times M$ matrix of independent identically distributed, standard Gaussian random

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

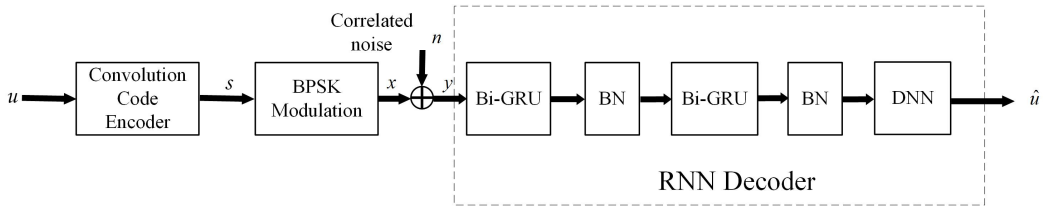


Fig. 3. The proposed channel decoding structure with the RNN decoder.

variables. The $(1/M)trace(C) = 1$ is required to keep the power of noise constant. The correlation matrix C of standard noise correlation model mentioned in the literature [1] is given by :

$$C_{ij}=\begin{cases} c^{j-i} & ,i<j \\ (c^{i-j})^{*} & ,i\geq j \end{cases} \tag{2}$$

where C_{ij} is the (i,j)th element of C and c is the correlation coefficient with $|c| \leq 1$.

At the receiver, the symbols are obtained by

$$y = x + n_c \tag{3}$$

The received channel value y is directly used for channel decoding. The RNN decoder decodes with a block of code of length M. Bidirectional GRU considers the symbols of length M as M time step input and it extracts sequence information through forward and reverses. The output of GRU will compress the input symbols into a vector of smaller dimensions and filters out certain noise. Then, the fully connected network obtain the decoding result $\hat{u}$ according to GRU output.

B. Parameter Settings and Training

Based on GRU and fully connected network, the design of the RNN decoder mainly depends on parameter settings, which directly affects decoder performance. The proposed design can be summarized using the relevant parameters, which will result in the tradeoff between performance and complexity. The RNN decoder parameter settings are summarized in Table I. Firstly, 2 important parameters are the number of layers and output dimensions of the GRU network. In the natural language processing where RNN is widely used, the number of layers and output dimensions of the network are generally large to ensure full extraction of complex information in the text. Note that although the general RNN framework may result in better performance, it will increase computational complexity and consume a great amount of resource in training and implement. The designed RNN decoder in Figure.3 aims to deal with convolutional codes after channel transmission which has small dimensions and amount of information, so the simple RNN model which contains 2 layers of GRU and 1 layer of the fully connected network is still suitable as a decoder.

After designing RNN framework, the training of the model requires many settings. In our experiments, we use a code block of length 100 as the input to the RNN decoder. The model parameters are initialized using random initialization. We choose mean squared error (MSE) as the loss function to train the RNN model. The goal of the training process is to minimize the loss.

TABLE I
RNN DECODER SETTINGS

RNN input data size	$n \times 100$
GRU Output size	200
GRU activation function	tanh
DNN activation function	sigmoid
Loss function	MSE
Total parameter number	968401

$$f_{\ell}(u,v) = |u-v|_2^2 \tag{4}$$

The training data needs to be generated before training the RNN model. In the practical use of network training, the validation data is generated to test the capability of network and void overfitting. We use single SNR (measured as E_b/N_0) training data to train the network. Our training of the network is implemented in Keras [18], using tensorflow [19] as a backend. The neural network runs on fast concurrent GPU architecture, which speeds up learning. Compared with different RNN decoders trained by single and multiple data from $\{0,1,2,3,4,5,6\}\,dB$, we found training model based on $0dB$ training data presents slightly better BER performance on the uniform validation data containing different SNRs.

We use mini-batch gradient descent method to train the network. Each mini-batch contains 200 blocks of training data, and the network updates the parameters one time based on one mini-batch. We adopt Adam optimization [20] method to adjust optimal network parameters. In addition, the learning rate gradually decreases with training. We check the loss value over the validation set for each epoch, in which the gradient of the loss function is calculated over the entire training set. In addition, the early stopping (training will stop if the loss of validation does not drop in 10 epochs) is used to against the overfitting. The RNN decoder training settings are summarized in Table II.

IV. SIMULATION RESULTS

In this section, we present the simulated and analytical results of the RNN decoder. The system performance is measured by the BER. In order to ensure the accuracy of BER, 10^8 information bits are tested for each SNR. Besides, the test set and training set are independent of each other. We focus

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TRAINING SETTINGS

Mini-batch size	200
Num of epoch	100
Num of the training data	50000
Num of the validation data	50000
SNR for generating the training data	0dB
Initialization method	Random initialization
Optimization method	Adam optimization
Initial learning rate	0.001
Num of early stopping	10
Trainable parameter number	966801

on the performance where $BER = 10^{-4}$. We evaluate the RNN decoder with different noise correlation levels, different correlation models and different convolutional codes. In order to support reproducible research, we provide the source code of this paper[1].

A. (2,1,3) convolutional code performance under different correlations

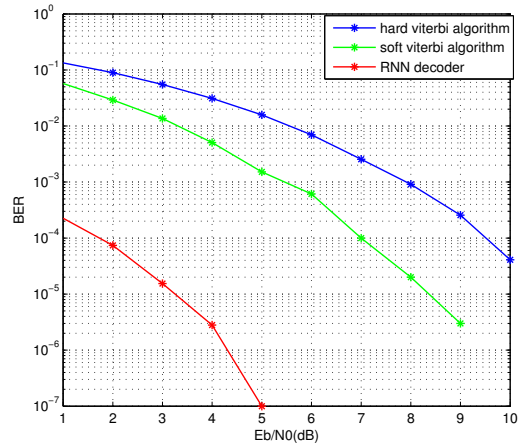
We first compare the performances of the RNN decoder and the Viterbi Algorithm decoder based on (2, 1, 3) convolutional code. Our simulation results are based on 3 correlation parameters: $c = 0.8$ means a relatively strong correlation model, $c = 0.5$ means a moderate correlation model and $c = 0$ means AWGN channel with no correlation.

Figure.4 shows the BER performance of the RNN decoder under different noise parameters of the (2, 1, 3) convolutional code. In order to display the results better, the BER performances of the hard decision and soft decision Viterbi decoding algorithm are also given. Figure.4(a) shows the performance of the RNN neural network decoder under strong correlation noise. The RNN decoder achieves 5dB performance improvement compared with the traditional soft decision Viterbi decoder and more than 7dB performance improvement compared with the traditional hard decision Viterbi decoder.

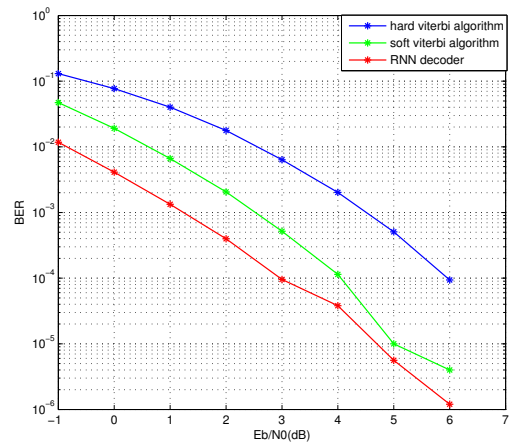
Figure.4(b) shows the decoding performance of the RNN decoder under moderate correlation noise. The RNN decoder achieves 1dB performance improvement compared with the traditional soft decision Viterbi decoder and 3dB performance improvement is achieved compared with the traditional hard decision Viterbi decoder. It should be emphasized that the RNN decoder can denoise the reception according to the correlation between the noises. The stronger correlated noise, the better denoising effects, so the RNN decoder obtains better decoding performance.

Figure.4(c) shows the decoding performance of RNN decoder under AWGN channel. The performance of the RNN neural network decoder is close to the performance of the soft decision Viterbi decoder. The original input 0, 1 bit probability is the same, so the soft decision Viterbi decoder

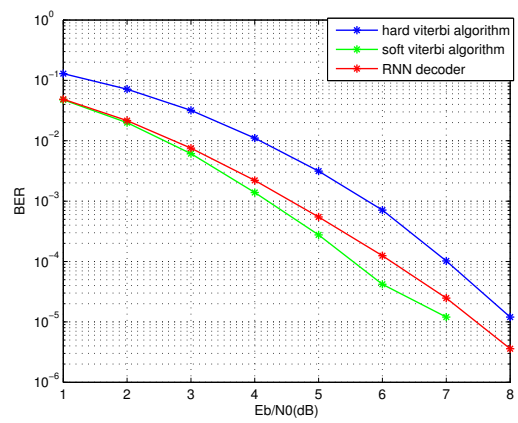
[1]Code is available at https://github.com/jimmyxiang1314/RNN-Decoder-Correlated-Noise.



(a) $c = 0.8$



(b) $c = 0.5$



(c) $c = 0$

Fig. 4. Performance of RNN decoder for (2, 1, 3) convolutional code

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.

is close to the MAP performance, which indicates that the performance of RNN decoder is close to that of MAP. This is because the RNN neural network decoder can fully extract the correlation information according to the correlation of the convolutional code through the bidirectional GRU structure under the AWGN channel.

We thus conclude that the traditional Viterbi decoding algorithm degrades performance under correlated noise. The stronger noise correlation, the lower decoding performance for the Viterbi decoding algorithm. Instead, the RNN neural network decoder can make full use of the correlation between noises to obtain better decoding performance. The stronger correlated noise, the greater improvement of the RNN decoder will be achieved compared to the traditional Viterbi decoders.

B. (2,1,3) convolutional code performance under pink noise

The RNN decoder support not only the specific correlation model, but also the real correlated noise. In this part, we will present the decoding performance under pink noise and show the robustness of the RNN decoder under different correlation models. As for different noise correlation models, the RNN decoder works through re-training the network. The pink noise can be described by the following power spectrum density:

$$P(f) \propto \frac{1}{|f|^{\alpha}}$$

when $\alpha = 1$, the noise is commonly referred to as pink noise. The simulation results are given in Figure.5. The RNN decoder achieves 3.5dB performance improvement compared with the traditional soft decision Viterbi decoder and more than 5dB performance improvement compared with the traditional hard decision Viterbi decoder. Clearly, the RNN decoder still achieves significant BER performance, indicating that the decoder is suited for different models.

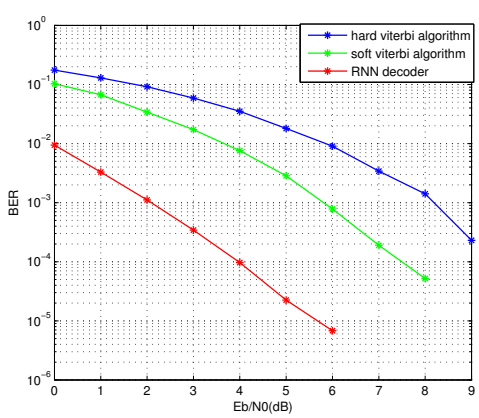


Fig. 5. RNN decoder performance under pink noise

C. Performance Under Different Convolutional Codes

In this section, we analyze the performance of different convolutional codes. Comparing (2,1,3), (2,1,4), (2,1,5), (2,1,6) and (2,1,7) convolutional codes, we obtain their BER performance under correlated noise. They have same convolutional code rates but different coded memory depths. The longer memory depth of the convolutional code, the better performance of the traditional Viterbi decoder. We will analyze the applicability and robustness of the RNN decoder based on the simulation results.

The performance of the RNN decoder is shown in Figure.6. Figure.6(a) shows the RNN decoder BER performance of different convolutional codes under strongly correlated noise. It can be seen that the decoder obtains best decoding performance on (2,1,3) convolutional code. The longer memory depth, the worse performance. When the memory depth reaches 7, the RNN decoder hardly decodes the received signals with different SNR effectively.

Figure.6(b) shows the BER performance of different convolutional codes on the RNN neural network decoder under moderate correlated noise. The performance trend is consistent with that under strongly correlated noise.

In summary, the performance of the RNN decoder decreases with the longer memory depths of the convolutional code. The RNN decoder can't learn a better decoding structure, so the more complex models are required for the convolutional codes with long memory depths. Due to the hardware constraints, the design and training of more complex models cannot be completed.

V. CONCLUSIONS

In this paper, we apply the GRU network and fully connected network to the decoder of the convolutional codes, and then propose the RNN decoder under correlated noise. The RNN decoder is better than the traditional Viterbi decoder for convolutional codes with shorter memory depths, such as (2, 1, 3) convolutional code under correlated noise. The greater noise correlation, the greater performance improvement of the decoder compared with the traditional Viterbi decoders. Besides, the RNN decoder is robustness under different correlation models. This network is convenient to use and we just need to retrain the RNN model for different situations. Due to the structure and complexity of the RNN decoder, as for longer memory depths of the convolutional codes, the decoding performance degrades gradually.

Furthermore, future investigations will focus on the design of more applicable models suited for all kinds of convolutional codes and low computational complexity.

REFERENCES

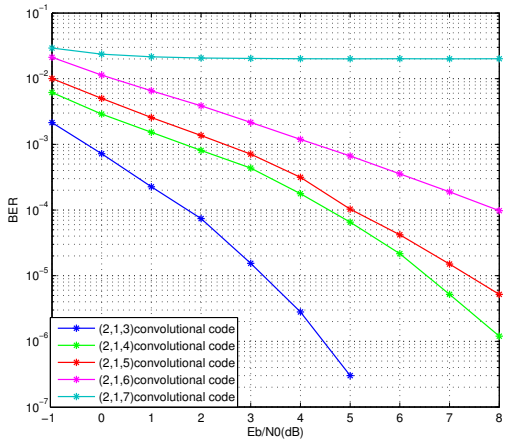
[1] S. K. Sharma, S. Chatzinotas and B. Ottersten, "SNR Estimation for Multi-dimensional Cognitive Receiver under Correlated Channel/Noise," in IEEE Transactions on Wireless Communications, vol. 12, no. 12, pp. 6392-6405, December 2013.

[2] A. Hajimiri and T. H. Lee, "A general theory of phase noise in electrical oscillators," in IEEE Journal of Solid-State Circuits, vol. 33, no. 2, pp. 179-194, Feb. 1998.

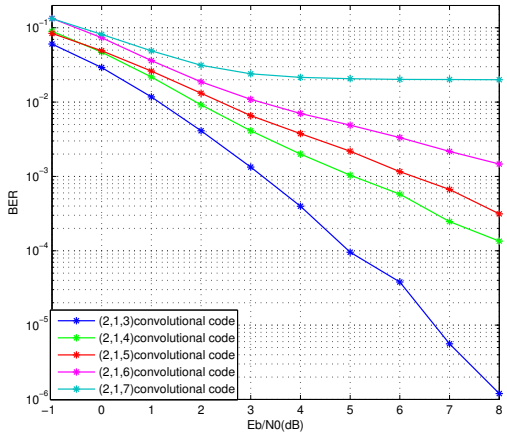
[3] Kay, S M . "Efficient generation of colored noise." Proceedings of the IEEE 69.4(2005):480-481.

[4] L. Di Bert , P. Caldera, D. Schwingshackl, and A. M. Tonello, "On noise modeling for power line communications," in Proc. IEEE Int. Symp. Power Line Commun. Appl., 2011, pp.283–288.

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.



(a) $c = 0.8$



(b) $c = 0.5$

Fig. 6. Performance of RNN decoder under different convolutional codes

[5] J. D. Wang and H. Y. Chung, "Trellis coded communication systems colored noise and the swapping technique," IEEE Trans.Commun., vol. 38, no. 9, pp. 1549–1556, Sep. 1990.

[6] T. Wang, C. K. Wen, H. Wang, F. Gao, T. Jiang and S. Jin, "Deep learning for wireless physical layer: Opportunities and challenges," in China Communications, vol. 14, no. 11, pp. 92-111, Nov. 2017.

[7] Lecun, Y , Y. Bengio , and G. Hinton . "Deep learning. " Nature 521.7553(2015):436.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Mach. Learn., 2015, pp. 448–456.

[9] T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," in IEEE Transactions on Cognitive Communications and Networking, vol. 3, no. 4, pp. 563-575, Dec. 2017.

[10] H. He, C. K. Wen, S. Jin and G. Y. Li, "Deep Learning-based Channel Estimation for Beamspace mmWave Massive MIMO Systems," in IEEE Wireless Communications Letters.

[11] W. Lyu, Z. Zhang, C. Jiao, K. Qin and H. Zhang, "Performance Evaluation of Channel Decoding with Deep Neural Networks," 2018 IEEE International Conference on Communications (ICC), Kansas City, MO, 2018, pp. 1-6.

[12] T. Gruber, S. Cammerer, J. Hoydis and S. t. Brink, "On deep learning-based channel decoding," 2017 51st Annual Conference on Information Sciences and Systems (CISS), Baltimore, MD, 2017, pp. 1-6.

[13] S. Cammerer, T. Gruber, J. Hoydis and S. ten Brink, "Scaling Deep Learning-Based Decoding of Polar Codes via Partitioning," GLOBECOM 2017 - 2017 IEEE Global Communications Conference, Singapore, 2017, pp. 1-6.

[14] F. Liang, C. Shen and F. Wu, "An Iterative BP-CNN Architecture for Channel Decoding," in IEEE Journal of Selected Topics in Signal Processing, vol. 12, no. 1, pp. 144-159, Feb. 2018.

[15] T. Wang, C. Wen, S. Jin and G. Y. Li, "Deep Learning-Based CSI Feedback Approach for Time-Varying Massive MIMO Channels," in IEEE Wireless Communications Letters, vol. 8, no. 2, pp. 416-419, April 2019.

[16] S. Dörner, S. Cammerer, J. Hoydis and S. t. Brink, "Deep Learning Based Communication Over the Air," in IEEE Journal of Selected Topics in Signal Processing, vol. 12, no. 1, pp. 132-143, Feb. 2018.

[17] B. Karanov et al., "End-to-End Deep Learning of Optical Fiber Communications," in Journal of Lightwave Technology, vol. 36, no. 20, pp. 4843-4855, 15 Oct.15, 2018.

[18] Keras Documentation Website: https://keras.io

[19] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Oper. Syst. Des. Implementation. Savannah,GA, USA, 2016.

[20] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," inProc. Int. Conf. Learn. Represen., 2015.

Authorized licensed use limited to: University of Southern California. Downloaded on April 02,2026 at 03:03:36 UTC from IEEE Xplore. Restrictions apply.